

PLR_UED 算法实现说明

对应本地运行：run_20260305_092522_719454_R30_0_10_90_PLR_UED_S42

文档范围 本文只描述当前项目中 PLR_UED 这一本地化实现在上述具体 run 中实际执行的算法流程、数学形式、伪代码、参数来源和运行产物。本文不讨论外部论文版本是否等价，也不对算法优劣作结论性评价。

代码根目录：A:\MYpython\34959_RL

文档日期：2026 年 3 月 7 日

目录

1	对象与边界	3
1.1	本说明对应的唯一运行实例	3
1.2	“PLR_UED”在本项目中的含义	3
2	项目内的整体算法结构	3
2.1	四阶段总流程	3
2.2	这次 run 的实际阶段数量	4
3	这次 run 的实际配置来源	4
3.1	Profile 显式给出的参数	4
3.2	Pipeline 默认参数带入的设置	5
3.3	内层算法与最终评测配置	5
4	本地化 action space 与 level 定义	6
4.1	level 参数化	6
4.2	action space v1	6
4.3	相位过滤	6
5	PLR replay buffer 的本地实现	6
5.1	buffer 条目结构	6
5.2	priority 更新	7
5.3	replay 抽样权重	7
5.4	new / replay 混合	7
6	动作选择流程的本地细节	8
6.1	这次 run 并非“TS 直接选动作”	8
6.2	伪代码：本地 outer 动作选择	8
7	本次 run 的 objective 公式	8
7.1	统计量定义	8
7.2	PLR objective 的真实实现	9
7.3	说明	9
8	Phase2 与 Phase3 的本地差异	9
8.1	Phase2: 固定 phase1 checkpoint, 纯 outer batch	9
8.2	Phase3: 链式 checkpoint + mixed curriculum	10
9	Phase3 curriculum 的本地实现	10

9.1	alpha 调度	10
9.2	文件级混合规则	10
9.3	本次 run 的实际课程混合统计	11
10	运行期 replay 统计	11
10.1	最终 replay 统计	11
10.2	buffer 文件中可见的 level 统计	11
11	本次 run 的完整伪代码	12
12	本次 run 的最终实现结果	13
12.1	最终 implement 汇总	13
12.2	这次 run 可以客观确认的实现特征	13
13	与抽象“PLR/UED”表述相比的本地化差异	13
14	附录：关键代码与结果文件索引	14

1 对象与边界

1.1 本说明对应的唯一运行实例

本文对应的运行目录为：

```
A:\Mypython\34959_RL\codes\nexus\local_adaptive_o10_90_s42_rerun2\
run_20260305_092522_719454_R30_0_10_90_PLR_UED_S42
```

该目录中可直接核对的关键文件包括：

- meta.json: 最终 implement 评测元信息；
- post_stage/pipeline_summary.json: phase1 / outer / phase4 串联摘要；
- post_stage/outer_train_round.csv: outer 每轮训练结果；
- post_stage/outer_actions.csv: outer 每轮选中的 level 参数；
- post_stage/outer_plr_stats.csv: PLR replay 统计；
- post_stage/outer_curriculum.csv: phase3 课程混合记录；
- post_stage/outer_policy_plr_buffer.json: 最终 replay buffer 状态；
- rl_summary.csv: 最终 implement 评测汇总。

1.2 “PLR_UED” 在本项目中的含义

在当前代码库中，PLR_UED 不是一个独立仓库，而是 Dynamic_master34959.py 中的一个 outer profile 名称。该 profile 会调用：

- codes/outer_rl/run_edrl_pipeline.py: phase1 → outer → phase4 的总调度；
- codes/outer_rl/run_edrl_phase2.py: outer 主循环；
- codes/Dynamic_master34959.py: phase1 与 phase4 的内层训练/评测入口；
- codes/core/dynamic_RL34959.py: 内层 RL 主流程；
- codes/algorithms/ppo_new/nn/context_extractor.py: PPO_NEW v3 表征主干。

因此，本文中的“PLR_UED”应理解为：当前项目内，以 PPO_NEW v3 为 inner learner、以 objective_mode=plr 和 level replay 为 outer 机制的本地实现版本。

2 项目内的整体算法结构

2.1 四阶段总流程

这次 run 的执行结构可以写成：

Phase1 warmup → Phase2 outer search → Phase3 curriculum + replay → Phase4 implement eval.

结合运行产物，可对应为：

1. **Phase1:** 在基础分布上训练一个 PPO_NEW v3 初始模型，产出 `theta_phase1.zip`;
2. **Phase2:** 以 `theta_phase1.zip` 为固定输入 checkpoint，逐轮生成 outer level，训练内层模型并以 `plr` 目标给 level 打分;
3. **Phase3:** 在上一轮 checkpoint 基础上继续迭代，同时把 outer level 生成的数据与 phase1 数据按比例混合成课程数据集;
4. **Phase4:** 用某个 outer 阶段 checkpoint 作为初始模型，在 `implement` 数据上做 `eval_only`，得到最终 `rl_summary.csv`。

2.2 这次 run 的实际阶段数量

根据 `outer_train_round.csv`:

- 总 outer 迭代数: 51;
- 其中 `phase2` = 21 轮;
- 其中 `phase3` = 30 轮。

根据 `pipeline_summary.json`:

- `phase1` checkpoint 为 `theta_phase1.zip`;
- `phase4` 初始化 checkpoint 为 `theta_iter051.zip`;
- `phase4` 实际执行了 `implement` 评测。

3 这次 run 的实际配置来源

3.1 Profile 显式给出的参数

在 `Dynamic_master34959.py` 中，PLR_UED profile 对 outer pipeline 显式添加了如下参数:

```
--outer-policy-mode ts
--objective-mode plr
--plr-level-replay
--plr-p-new 0.5
--plr-buffer-size 200
--plr-priority-ema-alpha 0.6
--plr-min-weight 0.05
--outer-curriculum-enable
```

这一点说明: 该 profile 的 outer 核心由三部分组成:

1. `objective_mode=plr`;
2. `plr-level-replay`;
3. `phase3` 课程混合 (`outer-curriculum-enable`)。

3.2 Pipeline 默认参数带入的设置

由于 PLR_UED profile 没有覆盖 outer pipeline 的全部参数，因此本次 run 还继承了 pipeline 的默认值。与本次实现直接相关的默认项如下：

参数	本次 run 的值
outer-phase	auto
outer-action-space-version	v1
outer-mu-choices	{10, 30, 60, 90}
outer-ratio-choices	{0.2, 0.3, 0.5, 0.7, 0.8}
outer-num-file-choices	{5, 10, 15}
outer-pattern-choices	{ab, random_mix}
phase2-fixed-num-files	5
phase3-num-file-choices	{5, 10, 15}
phase2-min-iters / max-iters	5/40
phase3-min-iters / max-iters	10/50
curriculum-alpha-start / end	0.70/0.35
curriculum-alpha-horizon	25
curriculum-replay-ratio	0.20
curriculum-replay-max-iters	5
plr-dj-weight	1.0
plr-j-weight	0.1
path-penalty	2.0

3.3 内层算法与最终评测配置

根据 meta.json 和 pipeline 调度方式：

- phase1 和 phase4 的 inner 算法均为 PPO_NEW;
- algo_version=v3;
- 分布为 0_10_90;
- 请求数为 30;
- seed 为 42;
- generator workers 为 2。

4 本地化 action space 与 level 定义

4.1 level 参数化

在当前实现中，一个 outer level 可写成：

$$g = (\mu_A, \mu_B, p, n, \pi, s),$$

其中：

- μ_A, μ_B ：两类事件均值参数，对应代码中的 `mu_a, mu_b`；
- p ：比例参数，对应 `ratio_a`；
- n ：文件数预算，对应 `num_files`；
- π ：模式类型，对应 `pattern`，本次 run 中为 `ab` 或 `random_mix`；
- s ：迭代种子，对应 `seed`，由 `base seed` 与 `iter_id` 派生。

4.2 action space v1

代码中的 `v1` 动作空间由笛卡尔积构成：

$$\mathcal{G} = \{(\mu_A, \mu_B, p, n, \pi) \mid \mu_A, \mu_B \in \{10, 30, 60, 90\}, \mu_A \neq \mu_B, p \in \{0.2, 0.3, 0.5, 0.7, 0.8\}, n \in \{5, 10, 15\}, \pi \in \{\text{ab}, \text{ra}\}\}$$

由于 $\mu_A = \mu_B$ 被显式排除，总动作数为：

$$|\mathcal{G}| = 4 \times 3 \times 5 \times 3 \times 2 = 360.$$

4.3 相位过滤

虽然完整 action space 含 $n \in \{5, 10, 15\}$ ，但当前实现按相位过滤：

- **phase2**：强制 $n = 5$ ；
- **phase3**：允许 $n \in \{5, 10, 15\}$ 。

这意味着在本次 run 中，`phase2` 只在 (μ_A, μ_B, p, π) 上搜索，而 `phase3` 才重新开放 n 维度。

5 PLR replay buffer 的本地实现

5.1 buffer 条目结构

当前实现中的 replay buffer 存储为一个 JSON 文件，条目结构为：

```
{
  "signature": "10|60|0.50000000|5|ab",
  "action": {...},
  "score_ema": 0.1538,
```

```

"last_score": 0.1052,
"n_seen": 4,
"n_sampled": 3,
"last_iter": 40
}

```

其中 action 的 signature 为:

$$\sigma(g) = \text{mu_a}|\text{mu_b}|\text{ratio_a}|\text{num_files}|\text{pattern}.$$

需要注意的是: **iter seed 不进入 signature**, 因此 replay 的 level 同一性只由五元组 $(\mu_A, \mu_B, p, n, \pi)$ 定义。

5.2 priority 更新

当某个 level 在第 t 轮得到 objective score $S_t(g)$ 后, 若该 level 已存在于 buffer 中, 则:

$$q_t(g) = (1 - \alpha) q_{t-1}(g) + \alpha S_t(g),$$

其中本次 run 的 $\alpha = 0.6$ 。

若该 level 首次出现, 则直接插入:

$$q_t(g) = S_t(g), \quad n_{\text{seen}}(g) = 1, \quad n_{\text{sampled}}(g) = 0.$$

5.3 replay 抽样权重

当从 buffer 中 replay 时, 代码不是直接做 softmax, 而是使用平移后的线性权重:

$$w_i = \max(\varepsilon, q_i - \min_j q_j + \varepsilon),$$

其中本次 run 的 $\varepsilon = 0.05$ 。

归一化后得到 replay 抽样概率:

$$P(i \mid \text{replay}) = \frac{w_i}{\sum_j w_j}.$$

5.4 new / replay 混合

本次 run 采用混合采样:

$$\Pr(\text{new}) = p_{\text{new}} = 0.5, \quad \Pr(\text{replay}) = 0.5.$$

更精确地说, 代码中的规则为:

$$\text{use_new} = (|\mathcal{B}| = 0) \vee (U < p_{\text{new}}),$$

其中 $U \sim \text{Uniform}(0, 1)$, $\mathcal{B}$ 表示 replay buffer。

6 动作选择流程的本地细节

6.1 这次 run 并非“TS 直接选动作”

虽然 PLR_UED profile 设置了 `--outer-policy-mode ts`,但在当前代码实现中,一旦 `plr-level-replay` 为真,真正的动作选择首先进入 `_choose_action_plr_mixed`,而不是进入 TS 采样分支。

因此,本次 run 的主动作来源实际上只有三种:

1. **new**: 从完整 action space 随机抽取一个 action template;
2. **replay**: 从 replay buffer 按 priority 抽取;
3. **policy_filtered**: 若 new/replay 抽到的 action 不满足当前相位约束,则在允许集合中均匀随机重采样。

也就是说, `policy_mode=ts` 在本次 `plr-level-replay` 路径里主要是保留接口和日志字段,并不主导 outer action 的实际选择。

6.2 伪代码: 本地 outer 动作选择

```
Input: action space G, replay buffer B, phase phi, p_new

1. Build allowed set A(phi)
   - phase2: keep only actions with n = 5
   - phase3: keep only actions with n in {5,10,15}

2. If B is empty or rand() < p_new:
   sample g from G uniformly # source = new
else:
   sample g from B with shifted-linear priority # source = replay

3. If g not in A(phi):
   resample g uniformly from A(phi) # source = policy_filtered

4. Attach iter-specific seed to g
5. Generate outer batch for g
6. Run inner PPO_NEW-v3 on that batch
7. Compute objective score S(g)
8. Update replay buffer with S(g)
```

7 本次 run 的 objective 公式

7.1 统计量定义

对 outer 第 t 轮, 代码从新增 decision/path 记录中计算:

- J_t : 该轮新增 decision 的平均 reward, 即 `avg_reward`;

- $\Delta J_t = J_t - J_{t-1}$, 首轮定义为 0;
- $D_t = \frac{\text{missing_paths}_t}{\max(1, \text{path_total}_t)}$;
- 其中 $\text{path_penalty}=2.0$ 。

当前 run 的 path_missing 全程为 0, path_read_rate 全程为 1.0, 因此实际上:

$$D_t = 0, \quad \forall t.$$

7.2 PLR objective 的真实实现

本次 run 的 outer 目标为:

$$S_t(g) = w_{\Delta J} |\Delta J_t| + w_J J_t - \lambda_D D_t, \quad (1)$$

其中:

$$w_{\Delta J} = 1.0, \quad w_J = 0.1, \quad \lambda_D = 2.0.$$

由于本次 run 中 $D_t = 0$, 因此实际生效的是:

$$S_t(g) = |\Delta J_t| + 0.1 J_t.$$

7.3 说明

这一点非常重要: 本地 PLR_UED profile 的 outer objective 实际上是一个基于 $|\Delta J| + 0.1J$ 的学习进展型得分, 而不是当前代码中的 EDRL-v4 目标函数。

换言之, 本次 run 的 PLR/UED 核心由两部分构成:

1. 用 $|\Delta J| + 0.1J$ 给 level 打分;
2. 用 priority replay + phase3 curriculum 反复利用这些高分 level。

8 Phase2 与 Phase3 的本地差异

8.1 Phase2: 固定 phase1 checkpoint, 纯 outer batch

从 `outer_train_round.csv` 可以直接看到, phase2 的 `ckpt_in` 始终为:

`theta_phase1.zip`.

因此本次 run 的 phase2 不是 checkpoint 链式推进, 而是:

- 固定从 phase1 初始模型出发;
- 每轮只更换 outer batch;
- 用该轮得到的 objective 更新 replay buffer。

同时, phase2 明确不启用 curriculum 混合; 即使 profile 打开了 `outer-curriculum-enable`, 代码也规定:

```
phase2 never uses curriculum.
```

8.2 Phase3: 链式 checkpoint + mixed curriculum

进入 phase3 后, `ckpt_in` 变为上一轮的 `theta_iterXXX.zip`, 即:

$$\theta_t^{\text{in}} = \theta_{t-1}^{\text{out}}.$$

同时, 训练数据不再直接使用 `outer_batches/iter_XXX`, 而是先构造:

```
post_stage/outer_curriculum/iter_XXX
```

并将其作为 `external_data_root` 交给内层 PPO_NEW v3。

9 Phase3 curriculum 的本地实现

9.1 alpha 调度

curriculum 的 outer 比例由线性调度给出:

$$\alpha(t) = \alpha_{\text{start}} + (\alpha_{\text{end}} - \alpha_{\text{start}}) \cdot \min\left(1, \frac{t-1}{H-1}\right),$$

其中:

$$\alpha_{\text{start}} = 0.70, \quad \alpha_{\text{end}} = 0.35, \quad H = 25.$$

需要注意: 该调度使用的是全局迭代编号 t , 而不是 phase3 内部编号。因此本次 run 在 phase3 第一个迭代 (全局第 22 轮) 时, 已经有:

$$\alpha(22) = 0.39375,$$

而不是 0.70。

9.2 文件级混合规则

对某个 phase3 迭代, 若当前 `num_files` = n , 则代码对 $i = 0, \dots, n-1$ 逐个复制生成 mixed 数据文件。对于每个文件:

$$\Pr(\text{outer bucket}) = \alpha(t), \quad \Pr(\text{base bucket}) = 1 - \alpha(t).$$

若进入 outer bucket, 则进一步以 replay 比例 $r = 0.2$ 判断:

$$\Pr(\text{replay file} \mid \text{outer bucket}) = r, \quad \Pr(\text{current outer file} \mid \text{outer bucket}) = 1 - r.$$

因此每个文件的最终来源分布为：

$$\Pr(\text{replay}) = \alpha(t) \cdot r, \quad (2)$$

$$\Pr(\text{current outer}) = \alpha(t) \cdot (1 - r), \quad (3)$$

$$\Pr(\text{base}) = 1 - \alpha(t). \quad (4)$$

9.3 本次 run 的实际课程混合统计

根据 `outer_curriculum.csv`：

- phase3 共构造了 30 个 mixed curriculum 目录；
- 第一个 phase3 迭代 (iter 22)：

$$\alpha = 0.39375, \quad (\text{outer}, \text{base}, \text{replay}) = (4, 1, 0).$$

- 最后一个 phase3 迭代 (iter 51)：

$$\alpha = 0.35, \quad (\text{outer}, \text{base}, \text{replay}) = (1, 4, 0).$$

10 运行期 replay 统计

10.1 最终 replay 统计

根据 `outer_plr_stats.csv` 的最后一行：

- 最终 buffer size: 24;
- 总采样次数: 33;
- 其中 new: 13;
- 其中 replay: 20;
- 最终 replay ratio: 0.6061;
- 最近 20 轮 replay ratio: 0.6;
- top-10 覆盖率: 0.7;
- top-10 sample share: 0.6。

10.2 buffer 文件中可见的 level 统计

最终 `outer_policy_plr_buffer.json` 显示, buffer 中保存的是 level signature 与其 score EMA。示例条目：

- 10|60|0.5|5|ab: score_ema=0.1538, n_seen=4, n_sampled=3;
- 60|90|0.7|5|ab: score_ema=0.1660, n_seen=6, n_sampled=5;
- 60|30|0.7|5|random_mix: score_ema=0.1763, n_seen=5, n_sampled=1。

11 本次 run 的完整伪代码

```
Input:
    distribution = 0_10_90
    request_number = 30
    inner learner = PPO_NEW-v3
    profile = PLR_UED

Phase1:
    Train PPO_NEW-v3 on base distribution
    Save theta_phase1.zip

Outer loop (auto = phase2 then phase3):
    Initialize replay buffer B = {}
    For iter = 1 ... 51:
        Determine phase:
            iter 1..21 -> phase2
            iter 22..51 -> phase3

        Build allowed action subset:
            phase2: n = 5 only
            phase3: n in {5,10,15}

        Select outer level:
            if B empty or rand < 0.5:
                sample candidate uniformly from full action space
            else:
                sample candidate from B by shifted-linear priority
            if candidate not allowed in current phase:
                resample uniformly from allowed subset

        Materialize outer batch for current level

        If phase2:
            train data = current outer batch only
            ckpt_in = theta_phase1.zip
        Else if phase3:
            mix current outer batch with base data and replay files
            train data = mixed curriculum directory
            ckpt_in = previous theta_iter

    Run inner PPO_NEW-v3 training on train data
    Collect new decisions / path map rows
    Compute:
        J = average reward of new decisions
        dJ = J - previous J
        D = missing_paths / path_total
        score = |dJ| + 0.1 * J - 2.0 * D
```

```
Update replay buffer entry for current level by EMA(alpha=0.6)
Save theta_iterXXX.zip
```

Phase4:

```
Load theta_iter051.zip
Reuse phase1 data root in eval_only implement mode
Produce rl_summary.csv and rl_trace.csv
```

12 本次 run 的最终实现结果

12.1 最终 implement 汇总

根据 rl_summary.csv:

指标	数值
reward_count	200
average_reward	0.54
std_reward	0.4984
removal_action	9
removal_action_reward	8
removal_wait_action	187
removal_wait_action_reward	96
insertion_action	0
insertion_action_reward	0
insertion_non_action	4
insertion_non_action_reward	4

12.2 这次 run 可以客观确认的实现特征

从代码与运行文件联合可确认:

1. inner learner 是 PPO_NEW v3;
2. outer objective 实际为 $|dJ| + 0.1J - 2D$;
3. replay priority 使用 score EMA, 而非 softmax value function;
4. replay/new 混合概率固定为 0.5/0.5;
5. phase2 只搜索 $n = 5$ 的 level, 并固定从 theta_phase1.zip 出发;
6. phase3 才打开 mixed curriculum, 且其 α 已接近下界 0.35;
7. phase4 使用的是 theta_iter051.zip。

13 与抽象“PLR/UED”表述相比的本地化差异

为了避免把通用术语和本地实现混淆, 这里只做事实列举:

